



PROFESSIONAL MASTERS IN INFORMATION AND CYBER SECURITY (PMICS)

CSE 810: PROJECT ON CYBER SECURITY

INCEPTION STAGE • SEPTEMBER 2026

RedGrid

Dependency-Constrained UCB Exploration for Autonomous Penetration Testing

PRESENTED BY

Name	Class Roll	Reg No
Nishan Paul	50028	H-55
Md Rakibur Rahman	50040	H-49

SUPERVISOR

Dr. Md. Abdur Razzaque

Professor and Chair

Department of Computer Science and Engineering

University of Dhaka

Why This Matters

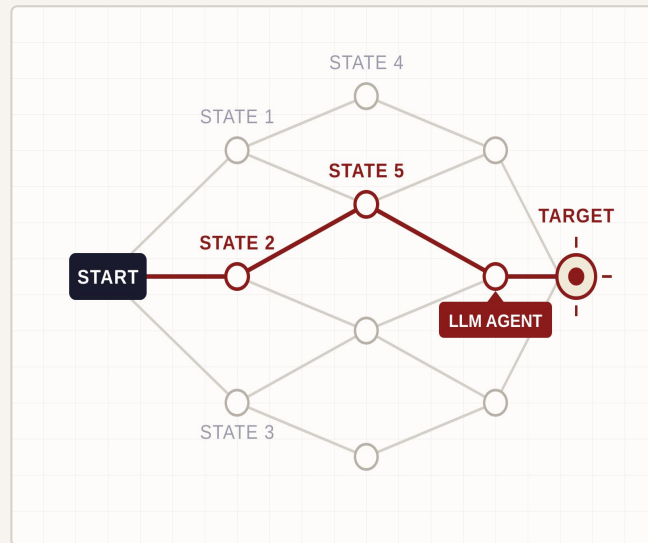
PROJECT MOTIVATION

*“Attackers move fast. Defenders move slow.
Automated testers don’t move at all.”*

Security vulnerabilities are found daily in web apps, APIs, and networks. Human pen testers are scarce and expensive. Automated scanners use fixed rule-sets — they cannot reason, adapt, or chain exploits.

The question:

Can an LLM agent carry out penetration testing autonomously — without step-by-step human direction?



ATTACK SURFACE EXPLORATION

Literature We Surveyed

A AGENT SYSTEMS

8 PAPERS

- | | | |
|----|--|---|
| 01 | AutoHack
Fang et al., 2024 | GPT-4 successfully exploits real web vulnerabilities |
| 02 | OneDay
Fang et al., 2024 | GPT-4 exploits one-day CVEs at 87% success with description |
| 03 | HPTSA
Zhu et al., 2024 | Hierarchical multi-agent system for zero-day CVEs |
| 04 | PentestGPT
Deng et al., 2024 | Context loss is the dominant failure in long sessions |
| 05 | VulnBot
Kong et al., 2025 | Role-specialised multi-agent dispatch architecture |
| 06 | CHECKMATE
Wang et al., 2025 | Classical planning integrated with LLM agent execution |
| 07 | Incalmo
Singer et al., 2025 | Multi-host orchestration across Active Directory |
| 08 | PrediQL
Liu et al., 2025 | Schema-aware LLM fuzzer for GraphQL APIs |

B BENCHMARKS

2 PAPERS

- | | |
|---|--|
| CVE-Bench
Zhu et al., 2025 | 40 critical web CVEs with oracle-backed evaluation |
| PentestEval
Yang et al., 2025 | Stage-level breakdown of pipeline failure modes |

S SURVEYS

1 PAPER

- | | |
|---------------------------------------|---|
| Wang 2025
Wang et al., 2025 | General survey of LLM-based agent architectures |
|---------------------------------------|---|

KEY TAKEAWAY

8 Agent Systems · 2 Benchmarks · 1 Survey

Benchmarks serve as the primary source for failure-mode analysis and motivate the research gap.

What the Literature Agrees On

"Architecture, not model scale, is the dominant variable."

CONSISTENT ACROSS

VulnBot

PentestGPT

CHECKMATE

Incalmo

HPTSA

Fang 2024

Unstructured ReAct + GPT-4	Structured Pipeline + GPT-4o-mini
X Depth-first tunnel vision	✓ Broad, structured exploration
X Context loss in long sessions	✓ Scoped per-invocation context
X No failure recovery	✓ Retry / adapt / escalate loops
X Implicit planning only	✓ Explicit dependency modeling

Synthesised from 6 independent papers | September 2026 Review

Failure Mode 1 — Insufficient Exploration

CVE-BENCH · 40 CRITICAL WEB CVEs · CVSS 9.0+

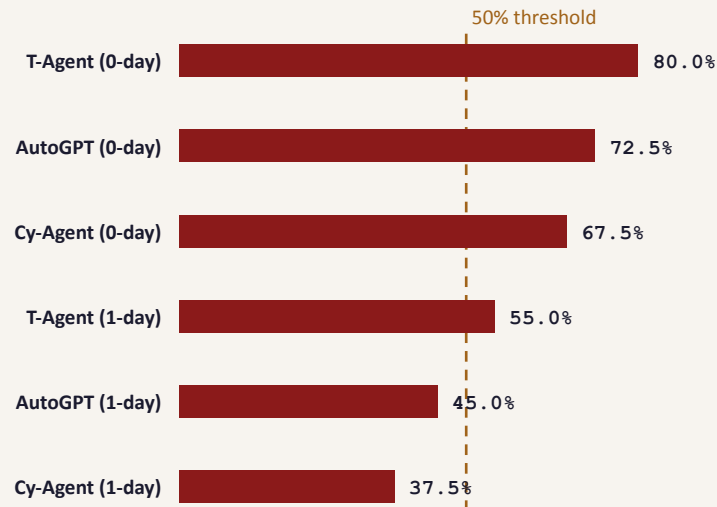
Even the best agent exploits only 13% (one-day) or 10% (zero-day).

The dominant failure is not reasoning quality — it is breadth of search. Agents commit early to a narrow attack path and never revisit.

Range: 37.5% – 80% exploration failure rates across all agents tested.

Root cause: Not a smarter-model problem — it's a search problem.

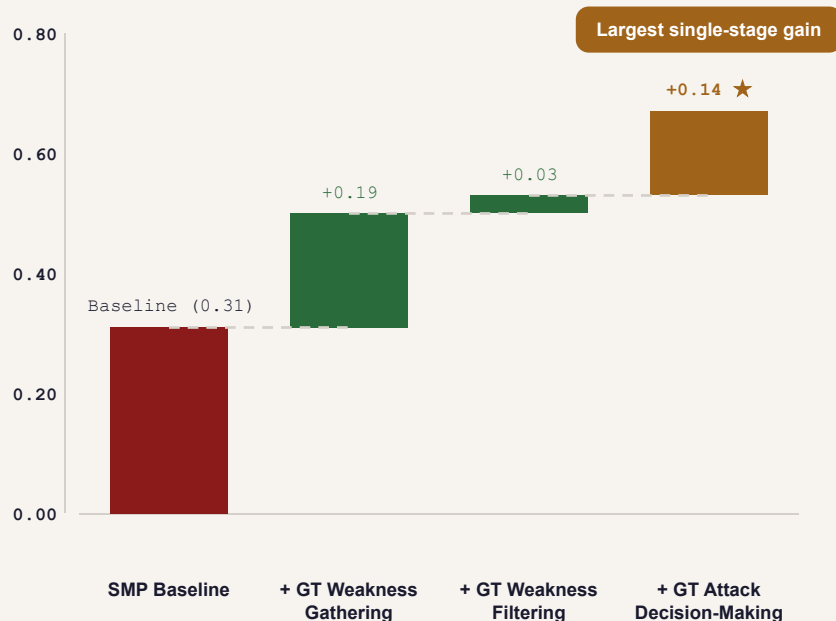
Exploration Failure Rate by Agent



Source: CVE-Bench (Zhu et al., 2025), Table 5

Failure Mode 2 — The Dependency-Reasoning Gap

PENTESTEVAL GT INJECTION ABLATION — TASK SCORE



Source: PentestEval (Yang et al., 2025)

PENTESTEVAL · 12 REAL-WORLD SCENARIOS · 346 TASKS

Attack Decision-Making (ADM) is the single weakest stage — Spearman $\rho = 0.25$.

Ground-truth ADM injection adds +0.14 on top of already-perfect weakness discovery. No existing system closes this gap.

0.25

ADM SPEARMAN ρ

+0.14

GT ADM GAIN

The Gap No System Has Closed

Wide Exploration

HPTSA VulnBot T-Agent AutoGPT

- ✓ Finds many candidate weaknesses
- ✗ No prerequisite / dependency model
- ✗ Flat task dispatch

UNEXPLORED TERRITORY

THE
COMPOUND
GAP

Dependency Reasoning

CHECKMATE PentestEval SMP

- ✓ Explicit prerequisite modeling
- ✗ Requires pre-enumerated weakness sets
- ✗ Does not scale to open-ended discovery

No system combines **open-ended exploration** with **dynamic dependency-aware planning**.

RedGrid investigates this exact combination.

Prior System Comparison

FOUR DIMENSIONS THAT MATTER MOST • PRELIMINARY 11-PAPER REVIEW

SYSTEM	ARCHITECTURE	DEPENDENCY MODELING	CROSS-SESSION MEMORY	BENCHMARK
Fang et al. 2024	Single agent (ReAct)	X None	X None	15 CVEs
HPTSA	Hierarchical planner + tasks	X Flat dispatch	X None	14 zero-day CVEs
PentestGPT	Split reasoning / parsing	~ Implicit (LLM only)	X None	HTB / VulnHub
VulnBot	Multi-agent, role-specialised	X Flat dispatch	X None	HTB-style
CHECKMATE	Agent + classical planner	✓ Explicit, pre-enumerated	X None	Curated only
Incalmo	Multi-host orchestration	~ Partial (host/cred)	X None	MHBench (40)
PrediQL	LLM-guided fuzzer	~ Schema-derived	X None	6 GraphQL APIs
RedGrid (proposed)	4-layer multi-agent	✓ Dynamic VDG	✓ 3-tier memory	Web + GraphQL + Multi-host

PRELIMINARY READING — 11-PAPER FOCUSED REVIEW, SEPTEMBER 2026

Research Gap

THE IDENTIFIED GAP

“The reviewed literature does not contain a system that combines:

- (1) broad, open-ended exploration of an unfamiliar attack surface,*
- (2) with an explicit, dynamically constructed dependency model,*
- (3) evaluated across more than one benchmarked attack-surface family.”*

Exploration Gap

37 – 80%

EXPLORATION FAILURE RATE

**CVE-Bench: insufficient exploration
is the dominant failure mode**

Planning Gap

0.25

ADM SPEARMAN p

**PentestEval: ADM is the weakest
stage across all pipelines**

Coverage Gap

1

ATTACK SURFACE PER SYSTEM

**Every prior system evaluated on
exactly one surface only**

This is the gap RedGrid is designed to investigate.

Introducing RedGrid

Dependency-Constrained UCB Exploration for Autonomous Penetration Testing

VDG

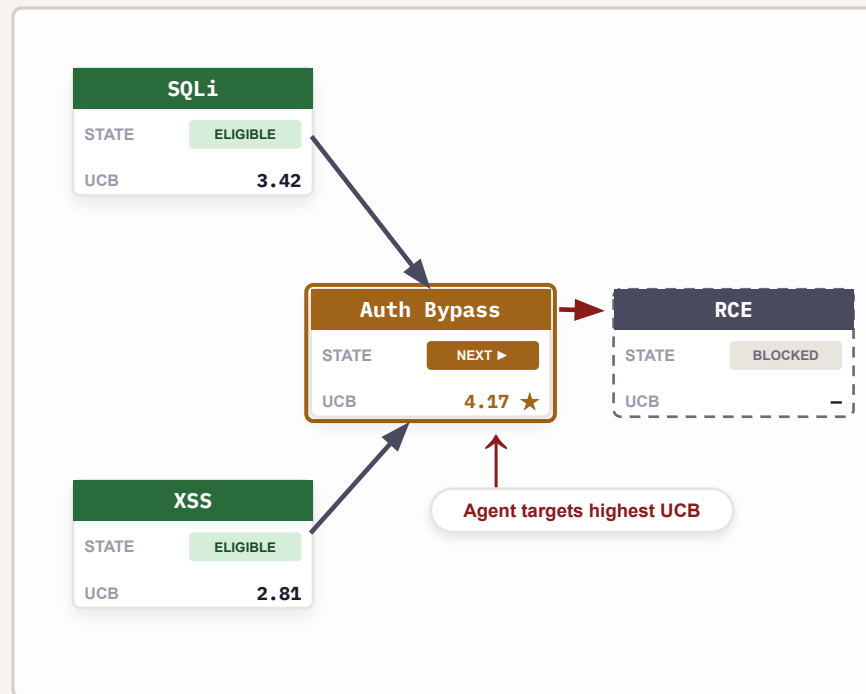
Model vulnerabilities as a graph with prerequisite edges, not a flat list.

UCB

Guide exploration with Upper Confidence Bound over the dependency-constrained frontier.

Memory

Retain and reuse strategies across missions.



Vulnerability Dependency Graph (VDG) – conceptual sketch

Research Objectives

FIVE WORKING GOALS AT INCEPTION STAGE

1

VDG Formalization

Investigate whether a **prerequisite-edge graph** improves agent exploration over a flat priority list

2

Dual-Layer World Model

Separate **confirmed facts** from **inferred hypotheses** for independent ablation

3

Multi-Layer Orchestration

Design a **four-layer agent hierarchy** matching the strongest surveyed systems

4

Cross-Mission Memory

Determine whether **strategy reuse** offers measurable benefit vs. **negative-transfer risk**

5

Benchmark-Grounded Evaluation

Evaluate on **oracle-backed benchmarks** across 3+ attack surfaces

NOTE

These reflect the research direction at inception stage. Scope will evolve with implementation.

Expected Contributions

ⓘ THE FOLLOWING SLIDES ARE HIGH-LEVEL SKETCHES — IMPLEMENTATION HAS NOT STARTED.

C1 — PRIMARY

Smarter Exploration

“Can modeling vulnerabilities as a graph — with explicit prerequisites — help an agent explore more intelligently?”

C2 — SUPPORTING

Learning Across Missions

“Can an agent carry forward what it learned in past engagements to perform better on similar targets?”

C3 — METHODOLOGICAL

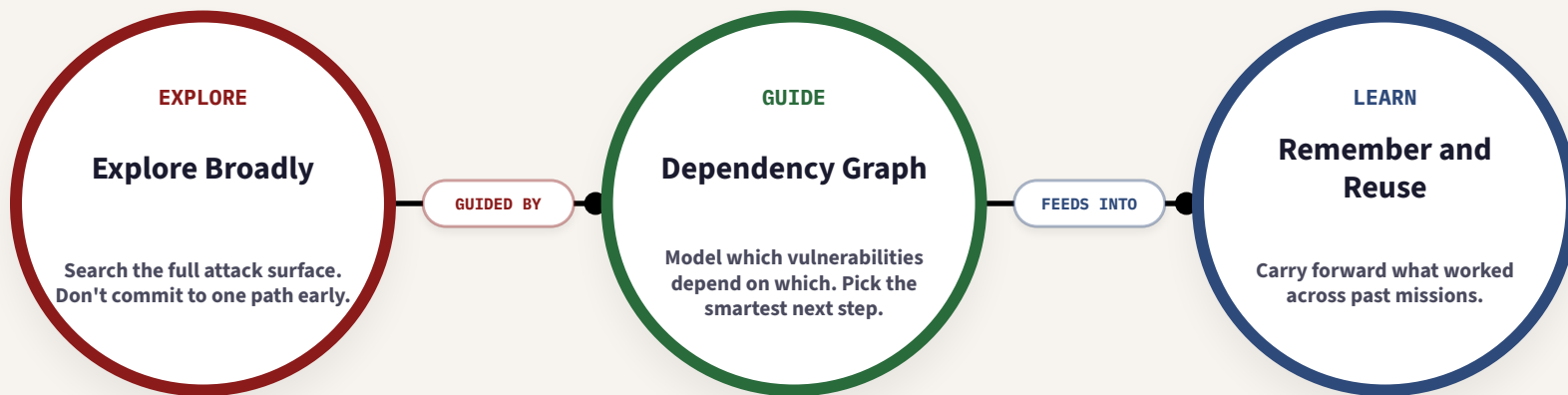
Consistent Evaluation

“Can a single architecture be tested fairly across web, GraphQL, and multi-host surfaces using standardized oracles?”

DISCLAIMER

These are research questions. Answers come from experiments — not this report.

RedGrid — The Core Idea



RedGrid asks whether connecting

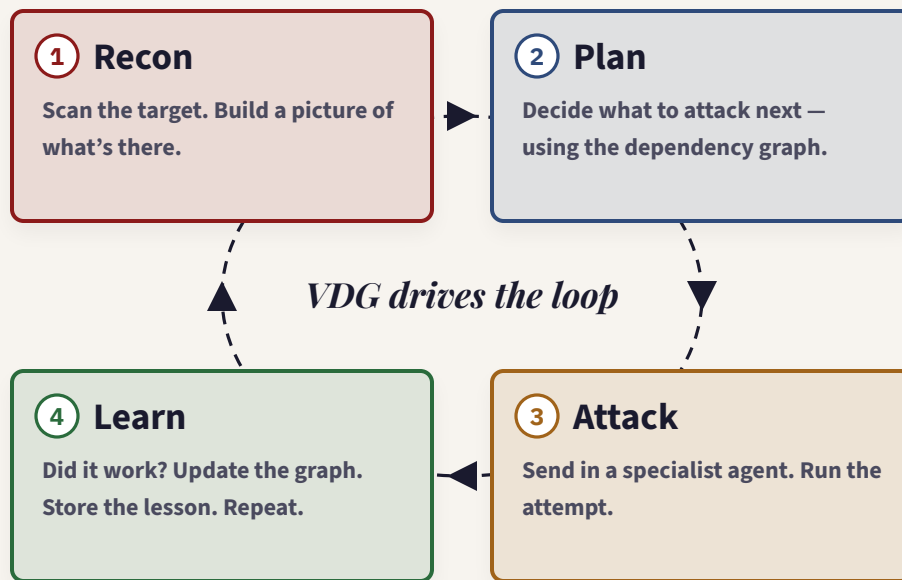


make autonomous penetration testing meaningfully better?

Architecture — A Rough Sketch



How RedGrid Will Work



01

Clean Slate per Attempt

Each attack attempt runs with fresh context — no leftover noise from previous steps.

02

Facts vs. Guesses

What the agent confirmed is kept strictly separate from what it infers.

Evaluation — Three Attack Surfaces

01. WEB APPLICATIONS

CVE-Bench

40 real critical CVEs

“Can the agent find and exploit web vulnerabilities without a hint?”

02. GRAPHQL APIs

PrediQL

6 real GraphQL APIs

“Can the agent abuse API schemas and find injection points?”

03. MULTI-HOST NETWORKS

MHBench

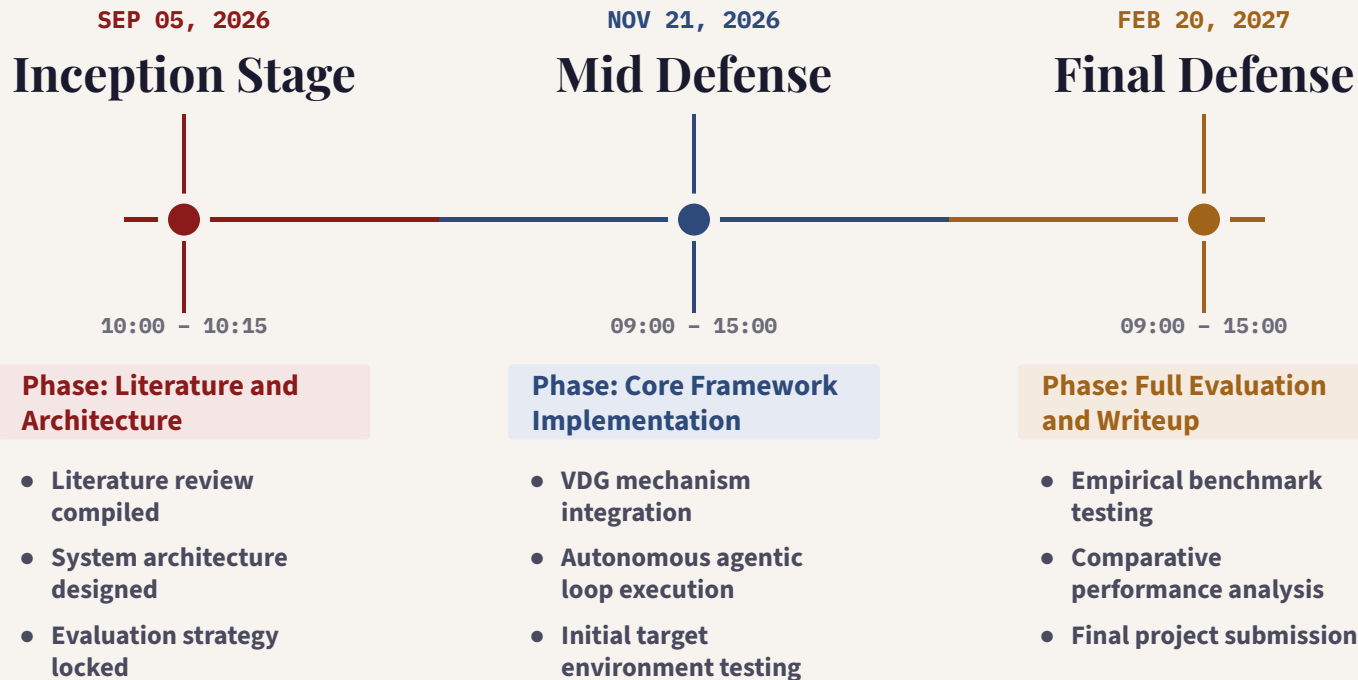
40 multi-host environments

“Can the agent move laterally across hosts and escalate privileges?”

We only test on attack surfaces that already have a published, oracle-backed benchmark.
No custom benchmarks.

Project Timeline

6-MONTH PROJECT • SEPTEMBER 2026 – FEBRUARY 2027



Challenges We Are Aware Of

ANTICIPATED BOTTLENECKS AND PROACTIVE MITIGATION

CRITICAL RISK

Inferring Dependencies Is Hard

Building the prerequisite graph using an LLM instead of human annotation will be noisy. We must measure this gap.

HIGH RISK

Sandbox vs. Real World

What works perfectly on a controlled benchmark may fail on a real-world target. We will test both environments.

MEDIUM RISK

Memory Could Backfire

A strategy that succeeds against software version 1 might backfire against version 2. A safety mechanism is key.

LOW RISK

Tuning Is Tricky

The system has parameters to adjust. Final results must be robust, and not depend on one perfect setup.

Where We Are

FOUNDATION SECURED

- Synthesized 11 key VAPT papers
- Identified 2 quantified failure modes
- Formalized the research gap
- Proposed 3 research directions
- Sketched the architecture approach
- Submitted inception report

The architectural blueprint is ready. Awaiting greenlight for implementation.

COMING NEXT

- 01 Build the prototype
- 02 Early experiments to test the ideas
- 03 Full benchmark evaluation (web, GraphQL, multi-host)
- 04 Refine based on what we find
- 05 Write the thesis

DISCUSSION POINT

Given the ambitious scope of VDG + dependency-aware exploration, should we pursue the full architecture, or constrain the scope to a specific subsystem before implementation?

RedGrid

Thank You

FLOOR OPEN FOR QUESTIONS

Nishan Paul and Md Rakibur Rahman

UNIVERSITY OF DHAKA

PROFESSIONAL MASTERS IN INFORMATION AND CYBER SECURITY (PMICS)

CSE 810: PROJECT ON CYBER SECURITY

Key References

- Fang, R., Bindu, R., Gupta, A., Zhan, Q., & Kang, D. (2024). *LLM agents can autonomously hack websites*. arXiv:2402.06664. **AutoHack**
- Fang, R., Bindu, R., Gupta, A., & Kang, D. (2024). *LLM agents can autonomously exploit one-day vulnerabilities*. arXiv:2404.08144. **OneDay**
- Zhu, Y., Kellermann, A., Gupta, A., Li, P., Fang, R., et al. (2024). *Teams of LLM agents can exploit zero-day vulnerabilities*. arXiv:2406.01637. **HPTSA**
- Deng, G., et al. (2024). *PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing*. **PentestGPT**
- Kong, L., et al. (2025). *VulnBot: Autonomous penetration testing for a multi-agent system*. **VulnBot**
- Wang, J., et al. (2025). *Automated penetration testing with LLM agents and classical planning*. **CHECKMATE**
- Singer, J., et al. (2025). *Incalmo: An autonomous LLM-assisted system for red teaming*. **Incalmo**
- Liu, Z., et al. (2025). *PrediQL: Automated testing of GraphQL APIs with LLMs*. **PrediQL**
- Zhu, Y., et al. (2025). *CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities*. **CVE-Bench**
- Yang, K., et al. (2025). *PentestEval: Benchmarking LLM-based penetration testing*. **PentestEval**
- Wang, L., Ma, C., Feng, X., et al. (2025). *A Survey on Large Language Model based Autonomous Agents*. **AgentSurvey**
- Sheyner, O., et al. (2002). *Automated generation and analysis of attack graphs*. IEEE S&P. **VDG**
- Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). *Finite-time Analysis of the Multiarmed Bandit Problem*. Machine Learning, 47(2-3). **UCB**
- Shinn, N., et al. (2023). *Reflexion: Language agents with verbal reinforcement learning*. NeurIPS. **ADM**